# Bibliotech — Route and Action Reference

**Project:** Bibliotech
**Document:** Complete HTTP route inventory, inputs, outputs and side effects
**Generated:** 2026-10-27
**Sources of truth:** `bibliotech/backend/server.js`, `bibliotech/backend/routes/*.js`, `bibliotech/backend/controllers/*.js`, `bibliotech/backend/application/services/*.js`, `bibliotech/backend/middleware/*.js`, `bibliotech/frontend/js/`

This document lists **61 API endpoints**, **21 frontend page routes**, static mounts and the conditional testing route. It reflects the code as implemented, including current edge cases.

# 1. How to Read This Document

Each endpoint entry contains:

- **Access:** public, optional authentication, any authenticated user, or role-restricted.
- **Input:** path/query/body/file fields actually read by the controller.
- **Success:** status and the main response fields. Response messages are localized text, not translation keys.
- **Errors and notes:** expected error statuses and important implementation behavior.

# 2. Global Conventions

## 2.1 Base URLs

| Mount | Purpose |
|---|---|
| `http://localhost:3000/api/auth` | Authentication, roles, user administration |
| `http://localhost:3000/api/careers` | Careers and year levels |
| `http://localhost:3000/api/subjects` | Subjects, subject materials, professor associations |
| `http://localhost:3000/api/materials` | Catalog, uploads, moderation, editing, favorites |
| `http://localhost:3000/api/subscriptions` | Student subscriptions and observer moderation |
| `http://localhost:3000/api/notifications` | In-app notification inbox |
| `http://localhost:3000/api/audit` | Audit trail (Superadmin) |
| `http://localhost:3000/api/thematic-tags` | Tag catalog and moderation |
| `http://localhost:3000` | Frontend page routes, static assets, `/uploads` |

## 2.2 Roles

| Role name in JWT/DB | Meaning in this document |
|---|---|
| `Superadmin` | Administrator. Referred to as “Admin” in tables. |
| `Profesor` | Professor. |
| `Alumno` | Student. |

## 2.3 Authentication

- Format: `Authorization: Bearer <JWT>`.
- JWT payload: `{ id, role }`; default expiry **2 hours** (`JWT_EXPIRES`).
- `verifyToken` only verifies the JWT signature and attaches `req.usuarioId` / `req.rol`. It does **not** re-query the user or re-check `estado`. A banned/deleted user with an existing JWT can keep using it until expiry; login itself rejects those accounts.
- `optionalAuth` attaches identity if a valid token is present and otherwise continues anonymously. An invalid token is silently ignored.
- Missing token → `401 { message }`.
- Wrong role → `403 { message }`.
- Public routes do not require a token; sending one may still enrich the response (for example, `esFavorito`).

## 2.4 Localization

Language resolution order:

1. `?lang=es|en`
2. `x-language` header
3. first value of `Accept-Language`
4. `DEFAULT_LANGUAGE` (default `es`)

Unknown languages fall back to `DEFAULT_LANGUAGE`. Controllers return translation keys; `ok()` and the error middleware resolve them into human-readable messages.

## 2.5 Response envelopes

Successful responses have no universal `data` wrapper. Examples:

```
{ "subjects": [ ... ] }
{ "materials": [ ... ] }
{ "message": "Material subido correctamente.", "material": { ... } }
```

Errors:

```
{ "message": "No autorizado.", "code": "errors.unauthorized" }
```

Validation errors:

```
{
  "message": "Datos invalidos.",
  "errors": ["El campo Titulo es obligatorio."],
  "fields": { "titulo": "El campo Titulo es obligatorio." }
}
```

Multer size/type errors may return only `{ "message": "..." }`.

## 2.6 Status codes

| Status | Meaning |
|---|---|
| `200` | Success. |
| `201` | Resource created (or duplicate tag returned through the create endpoint). |
| `400` | Invalid token/code, missing required data in a few services, Multer file errors, thematic tag invalid. |
| `401` | Missing/invalid JWT. |
| `403` | Authenticated but role or ownership check failed; account blocked at login. |
| `404` | Resource not found. |
| `409` | Duplicate email, pending-upload limit, duplicate file hash. |
| `422` | Field/business validation error. |
| `429` | Upload rate limit exceeded. |
| `500` | Unexpected server/DB error. |

## 2.7 Naming

- Most JSON uses **camelCase** (`createdAt`, `profesorId`, `fileHash`, `estadoAsociacion`). Exceptions: `GET /api/subscriptions/mine` returns raw `alumno_id`, `materia_id` and `created_at` alongside camel aliases; thematic-tag read responses return `creadoPor` / `createdAt` as `null` because of a mapper mismatch.
- Database columns use **snake_case** (`created_at`, `profesor_id`, `file_hash`, `estado_asociacion`).
- Enum values preserve Spanish labels (`Aprobado`, `Pendiente`, `Cancelado`, `Rechazado`, `Suscrito`, `Baneado`, `Activa`, `Pendiente`).

## 2.8 Pagination, filtering and sorting

- The API does **not** implement page/pageSize for most endpoints.
- The frontend paginates client-side with `Pagination.DEFAULT_PAGE_SIZE = 6`.
- Exception: `GET /api/materials/favorites/mine` passes `limit` and `offset` query values to the repository; values are parsed internally without explicit validation.
- Lists are usually newest-first (`created_at DESC`) or alphabetically by career/year/subject.

## 2.9 Upload limits and storage

| Limit | Value |
|---|---|
| Max file size per file | 100 MB (`MAX_FILE_SIZE`) |
| Max files per multipart request | 220 |
| Batch item limit | 100 |
| Cover image max size | 5 MB |

| Limit | Value |
| --- | --- |
| Cover extensions | `.jpg`, `.jpeg`, `.png`, `.webp`, `.gif` |
| Upload rate limit | 50 requests/minute per user/IP by default; disabled in `NODE_ENV=testing` |
| Local storage | `backend/uploads/`, served at `/uploads/...` |
| Cloud storage | Cloudinary when `FILES_MODE=cloud` |
| Duplicate detection | SHA-256 `file_hash`, unique per active material |

## 2.10 Document/file types

| Type | Accepted input |
| --- | --- |
| `PDF` | `.pdf` |
| `EPUB` | `.epub` |
| `Word` | `.doc`, `.docx` |
| `Presentacion` | `.ppt`, `.pptx`, `.odp` |
| `Link_Drive` | `drive.google.com` file/folder/open/uc URLs |
| `Youtube` | YouTube watch, shorts, embed, live, youtu.be |
| `Repositorio` | GitHub repository URLs |

## 2.11 Audit labels

Audit rows store human-readable labels, for example:

`Alta Usuario`, `Verificacion Cuenta`, `Solicitud Reseteo Clave`, `Cambio Clave`, `Desbaneo Usuario`, `Baja Usuario`, `Alta Rol`, `Solicitud Materia`, `Alta Materia`, `Asociacion Materia`, `Modificacion Materia`, `Alta Carrera`, `Modificacion Carrera`, `Alta Anio`, `Modificacion Anio`, `Subida Material`, `Asociacion Material`, `Aprobacion Material`, `Modificacion Material`, `Desasociacion Material`, `Cancelacion Material`, `Borrado Material`, `Suscripcion Materia`, `Baneo Alumno`.

## 2.12 Route-matching notes

- Material special paths (`/global`, `/author/:id`, `/mine`, `/pending`, `/pending-review`, `/favorites/mine`) are declared **before** `/:id` routes, so they are not captured as IDs.
- `/api/materials/:id/subjects/:subjectId` is declared before `/api/materials/:id`; Express matches the longer path first.
- `/api/careers/years/:yearId` is a separate two-segment path and is not shadowed by `/api/careers/:id`.
- The bare `DELETE /api/materials/:id` route is admin-only, but the controller also accepts `?subjectId=` and will perform a subject-specific removal.

# 3. Route Summary

## 3.1 Authentication and users — 17 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 1 | `POST` | `/api/auth/register` | Public | Register a student account. |
| 2 | `POST` | `/api/auth/verify-account` | Public | Verify/activate using body token/code. |
| 3 | `GET` | `/api/auth/verify-account` | Public | Verify/activate using query token/code. |
| 4 | `POST` | `/api/auth/activate` | Public | Activate using body token. |
| 5 | `GET` | `/api/auth/activate/:token` | Public | Activate using path token. |
| 6 | `POST` | `/api/auth/resend-verification` | Public | Resend activation email. |
| 7 | `POST` | `/api/auth/login` | Public | Authenticate and obtain JWT. |
| 8 | `POST` | `/api/auth/forgot-password` | Public | Request password reset. |
| 9 | `GET` | `/api/auth/reset-password/:token/validate` | Public | Check reset token validity. |
| 10 | `POST` | `/api/auth/reset-password` | Public | Set a new password. |
| 11 | `GET` | `/api/auth/roles` | Public | List roles. |
| 12 | `GET` | `/api/auth/me` | Any authenticated | Current safe user. |
| 13 | `POST` | `/api/auth/professors` | Superadmin | Create a professor account. |
| 14 | `GET` | `/api/auth/users` | Superadmin | List users. |
| 15 | `PUT` | `/api/auth/users/:id/ban` | Superadmin | Set user state `Baneado`. |
| 16 | `PUT` | `/api/auth/users/:id/unban` | Superadmin | Set user state `Activo`. |
| 17 | `DELETE` | `/api/auth/users/:id` | Superadmin | Soft-delete user (`Inactivo`). |

## 3.2 Careers and years — 6 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 18 | `GET` | `/api/careers` | Public/optional auth | List careers with years and counts. |
| 19 | `GET` | `/api/careers/:id` | Public/optional auth | Get one career with years. |
| 20 | `POST` | `/api/careers` | Superadmin | Create a career. |

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 21 | PUT | /api/careers/:id | Superadmin | Update a career. |
| 22 | POST | /api/careers/:id/years | Superadmin | Create a year/level in a career. |
| 23 | PUT | /api/careers/years/:yearId | Superadmin | Update a year/level. |

## 3.3 Subjects — 8 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 24 | GET | /api/subjects | Public/optional auth | List/filter subjects. |
| 25 | GET | /api/subjects/:id | Public/optional auth | Get one subject. |
| 26 | GET | /api/subjects/:id/materials | Public/optional auth | Approved materials for a subject. |
| 27 | POST | /api/subjects | Superadmin | Create a subject. |
| 28 | POST | /api/subjects/:id/request-association | Profesor | Professor requests assignment. |
| 29 | POST | /api/subjects/:id/confirm-association | Superadmin | Confirm/complete assignment. |
| 30 | POST | /api/subjects/:id/reject-association | Superadmin | Reject and clear assignments. |
| 31 | PUT | /api/subjects/:id | Superadmin | Update subject and professors. |

## 3.4 Materials — 16 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 32 | GET | /api/materials/global | Public/optional auth | Global approved catalog. |
| 33 | GET | /api/materials/author/:id | Public/optional auth | Public author profile + approved materials. |
| 34 | GET | /api/materials/mine | Alumno/Profesor/Superadmin | Own materials, any state. |
| 35 | GET | /api/materials/pending | Alumno/Profesor | Own pending materials. |
| 36 | GET | /api/materials/pending-review | Profesor/Superadmin | Pending suggestions in professor's subjects. |
| 37 | GET | /api/materials/favorites/mine | Alumno/Profesor/Superadmin | Current user's favorites. |

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 38 | POST | /api/materials/upload | Alumno/Profesor | Upload or link a material. |
| 39 | POST | /api/materials/batch | Superadmin | Batch upload/link materials. |
| 40 | POST | /api/materials/:id/approve | Profesor/Superadmin | Approve a pending association. |
| 41 | POST | /api/materials/:id/reject | Profesor/Superadmin | Reject a pending association. |
| 42 | POST | /api/materials/:id/associate | Profesor | Add material to owned subjects. |
| 43 | POST | /api/materials/:id/cancel | Alumno/Profesor | Cancel own pending material. |
| 44 | PUT | /api/materials/:id | Profesor/Superadmin | Edit a material. |
| 45 | DELETE | /api/materials/:id/subjects/:subjectId | Profesor/Superadmin | Remove material from one subject. |
| 46 | DELETE | /api/materials/:id | Superadmin | Delete globally or remove one association via query. |
| 47 | POST | /api/materials/:id/favorite | Alumno/Profesor/Superadmin | Toggle favorite. |

## 3.5 Subscriptions — 5 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 48 | GET | /api/subscriptions/mine | Alumno | Own subject subscriptions. |
| 49 | POST | /api/subscriptions/subscribe/:subjectId | Alumno | Subscribe to a subject. |
| 50 | DELETE | /api/subscriptions/subscribe/:subjectId | Alumno | Unsubscribe. |
| 51 | GET | /api/subscriptions/subject/:subjectId/observers | Profesor/Superadmin | Active observers. |
| 52 | POST | /api/subscriptions/ban/:alumnoId/:subjectId | Profesor/Superadmin | Ban an observer. |

## 3.6 Notifications — 3 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 53 | GET | /api/notifications | Any authenticated | Inbox + unread count. |
| 54 | POST | /api/notifications/read-all | Any authenticated | Mark all as read. |

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 55 | `POST` | `/api/notifications/:id/read` | Any authenticated | Mark one as read. |

### 3.7 Audit — 2 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 56 | `GET` | `/api/audit` | Superadmin | Enriched audit trail. |
| 57 | `GET` | `/api/audit/user/:id` | Superadmin | Raw logs by actor. |

### 3.8 Thematic tags — 4 routes

| # | Method | Path | Access | Purpose |
|---|---|---|---|---|
| 58 | `GET` | `/api/thematic-tags` | Public | List tags. |
| 59 | `POST` | `/api/thematic-tags` | Alumno/Profesor/Superadmin | Create a tag (or return existing). |
| 60 | `PUT` | `/api/thematic-tags/:id/ban` | Superadmin | Ban a tag. |
| 61 | `DELETE` | `/api/thematic-tags/:id` | Superadmin | Delete a tag. |

# 4. Detailed Endpoint Reference

## 4.1 Authentication and Users

### 1. `POST /api/auth/register`

**Access:** Public.

**Input (JSON):**

| Field | Type | Required | Rules |
|---|---|---|---|
| `nombre` | string | yes | 2–50 characters. |
| `apellido` | string | yes | 2–50 characters. |
| `email` | string | yes | Valid email; unique; normalized to lowercase. |
| `password` | string | yes | Minimum 8 characters, at least one letter and one digit. |
| `passwordConfirm` | string | no | If supplied, must equal `password`. |

**Action:** Creates an `Alumno` in `PendienteVerificacion`, hashes the password with bcrypt, creates a hashed verification token (default 60 minutes), attempts to send an activation email, and writes audit `Alta Usuario`.

**Success:** `201`

```
{
  "message": "Registro exitoso...",
  "user": { "id": 4, "nombre": "...", "email": "...", "estado": "PendienteVerificacion", "...":
"..." },
  "devToken": "optional only outside production"
}
```

**Errors:** `409 validation.emailTaken`; `422` field validation; `500` missing `Alumno` role. Mail failure is logged but does not fail registration.

### 2. `POST /api/auth/verify-account`

**Access:** Public.

**Input:** body may contain `token` or `code`, and optionally `email`.

- If `email` and token/code are both supplied, the token must belong to that email.
- With only token/code, the token alone is used.

**Success:** `200 { "message": "Cuenta verificada..." }`

**Errors:** `400 errors.invalidToken` for missing, expired, already-used, or mismatched token.

**Action:** Sets `users.estado = 'Activo'`, clears the verification token, records `Verificacion Cuenta`.

### 3. `GET /api/auth/verify-account`

Same as #2, but reads `token` / `code` / `email` from query parameters.

### 4. `POST /api/auth/activate`

**Access:** Public.

**Input:** `{ "token": "..." }`.

**Success:** `200 { "message": "Cuenta verificada..." }`

**Errors:** `400 errors.invalidToken`.

### 5. `GET /api/auth/activate/:token`

Same as #4 with the token in the path.

### 6. `POST /api/auth/resend-verification`

**Access:** Public.

**Input:** `{ "email": "..." }` (required, valid email).

**Action:** If the account exists and is pending, generates a new token and sends a new email. Otherwise returns the same generic message.

**Success:** `200 { "message": "..." }`, plus `devToken` outside production.

**Errors:** `422` invalid email.

**Privacy:** Does not reveal whether the email exists.

## 7. `POST /api/auth/login`

**Access:** Public.

**Input:**

| Field | Required |
|---|---|
| `email` | yes |
| `password` | yes |

**Action:** Finds user by normalized email, compares bcrypt hash, checks `estado`, signs JWT `{ id, role }` with 2-hour default expiry.

**Success:** `200`

```
{
  "message": "Sesion iniciada...",
  "token": "eyJ...",
  "user": { "id": 2, "roleNombre": "Profesor", "nombreCompleto": "...", "...": "..." }
}
```

**Errors:**

- `422` missing email/password.
- `401 errors.invalidCredentials` for unknown email or wrong password.
- `403 errors.accountNotVerified` for `PendienteVerificacion`.
- `403 errors.accountDisabled` for `Inactivo` / `Baneado`.

**Audit:** None on login.

## 8. `POST /api/auth/forgot-password`

**Access:** Public.

**Input:** `{ "email": "..." }`.

**Action:** Always returns a generic response. If the user exists, creates a hashed reset token (default 60 minutes), sends the recovery email, and records `Solicitud Reseteo Clave`. Audit failure does not block.

**Success:** `200`

```
{ "message": "...", "expiresInMinutes": 60, "devToken": "optional outside production" }
```

If the email does not exist, only the generic `message` is returned (no `expiresInMinutes` / `devToken`).

**Note:** The service does not filter by `users.estado`; an inactive or banned account can still receive a reset token, although login remains blocked.

**Errors:** `422` invalid email.

### 9. `GET /api/auth/reset-password/:token/validate`

**Access:** Public.

**Input:** token in path.

**Success:** `200 { "valid": true|false }`

**Notes:** Does not consume the token. `false` for missing/expired/unknown tokens. No error response for an invalid token.

### 10. `POST /api/auth/reset-password`

**Access:** Public.

**Input:**

| Field | Required | Rules |
|---|---|---|
| `token` or `code` | yes | Must map to a non-expired reset token. |
| `password` | yes | Minimum 8, letter + digit. |
| `passwordConfirm` | no | If supplied, must match. |

**Action:** Updates the bcrypt password, clears reset token fields, records `Cambio Clave`.

**Success:** `200 { "message": "Contrasena actualizada..." }`

**Errors:** `422` validation; `400 errors.invalidToken` for unknown/expired token.

### 11. `GET /api/auth/roles`

**Access:** Public.

**Success:** `200 { "roles": [ { "id": 1, "nombre": "Superadmin", "descripcion": "..." }, ... ] }`

### 12. `GET /api/auth/me`

**Access:** Any authenticated user.

**Success:** `200 { "user": { ...safe user... } }`

**Errors:** `401`; `404` if the user no longer exists.

### 13. `POST /api/auth/professors`

**Access:** Superadmin.

**Input:** Same fields as registration (`nombre`, `apellido`, `email`, `password`, optional `passwordConfirm`).

**Action:** Creates an account with role `Profesor` and default state `Activo`; hashes the password; records `Alta Usuario` with the admin as actor.

**Success:** `201 { "message": "...", "user": { ... } }`

**Errors:** `409` duplicate email; `422` validation.

**Note:** The professor is not assigned to any subject by this endpoint; use subject creation/association. Unlike registration/login, this path does not trim or lowercase the submitted email before insert.

### 14. `GET /api/auth/users`

**Access:** Superadmin.

**Success:** `200 { "users": [ ...safe users... ] }`

**Note:** Returns all users; passwords/tokens are omitted by the repository mapping.

### 15. `PUT /api/auth/users/:id/ban`

**Access:** Superadmin.

**Input:** none (path id).

**Action:** Sets `users.estado = 'Baneado'`; records `Baneo Alumno` (even for non-students).

**Success:** `200 { "message": "Usuario baneado..." }`

**Errors:** `404` user not found.

**Note:** There is no target-role or self-protection check; any existing account, including a Superadmin or the caller, can be banned. Existing JWTs remain usable until expiry.

### 16. `PUT /api/auth/users/:id/unban`

**Access:** Superadmin.

**Action:** Sets `users.estado = 'Activo'`; records `Desbaneo Usuario`.

**Success:** `200 { "message": "Usuario desbaneado..." }`

**Errors:** `404` user not found.

### 17. `DELETE /api/auth/users/:id`

**Access:** Superadmin.

**Action:** Soft delete: `users.estado = 'Inactivo'`; records `Baja Usuario`. The row is not deleted.

**Success:** `200 { "message": "Usuario eliminado..." }`

**Errors:** `404` user not found.

**Note:** Existing JWTs remain usable until expiry because `verifyToken` does not re-check state.

## 4.2 Careers and Years

### 18. `GET /api/careers`

**Access:** Public / optional auth (authentication does not change the payload).

**Input:** none.

**Success:** `200`


```
{
  "careers": [
    {
      "id": 1,
      "nombre": "Analista de Sistemas",
      "color": "#6f5bd3",
      "totalMaterias": 23,
      "totalMateriales": 0,
      "years": [
        { "id": 1, "careerId": 1, "nombre": "Primer Año", "orden": 1, "totalMaterias": 8,
"totalMateriales": 0 }
      ]
    }
  ]
}
```


**Notes:** Careers are ordered by name; years by `orden`, then name. Totals count subjects and distinct approved associations.

### 19. `GET /api/careers/:id`

**Access:** Public / optional auth.

**Success:** `200 { "career": { ...with years... } }`

**Errors:** `404 errors.notFound`.

### 20. `POST /api/careers`

**Access:** Superadmin.

**Input (JSON):**

| Field | Required | Rules |
|---|---|---|
| `nombre` | yes | 2–100 characters. |
| `color` | no | `#RRGGBB`; defaults to `#6f5bd3`; lowercased. |

**Action:** Inserts career; records `Alta Carrera`; returns the created career with years/counts.

**Success:** `201 { "message": "...", "career": { ... } }`

**Errors:** `422` invalid name/color; `404` not applicable; duplicate name is constrained by the DB and may surface as `500` because it is not translated into `409`.

### 21. `PUT /api/careers/:id`

**Access:** Superadmin.

**Input:** any of `nombre`, `color`; at least one required.

**Action:** Updates; records `Modificacion Carrera`.

**Success:** `200 { "message": "...", "career": { ... } }`

**Errors:** `404`; `422` no fields/invalid name/color.

### 22. `POST /api/careers/:id/years`

**Access:** Superadmin.

**Input:**

| Field | Required | Rules |
|---|---|---|
| `nombre` | yes | 2–100 characters. |
| `orden` | no | Coerced with `Number(value) \|\| 0`; defaults to current `years.length + 1`. Negative numbers are accepted and non-numeric values become `0`; there is no minimum check on create. |

**Action:** Inserts year in the career; records `Alta Anio`.

**Success:** `201 { "message": "...", "year": { ... } }`

**Errors:** `404` career; `422` invalid name; duplicate `(career_id, nombre)` may surface as `500`. `orden` itself has no create-time validation.

### 23. `PUT /api/careers/years/:yearId`

**Access:** Superadmin.

**Input:** any of `nombre`, `orden`; at least one required.

**Action:** Updates year; records `Modificacion Anio`.

**Success:** `200 { "message": "...", "year": { ... } }`

**Errors:** `404`; `422` no fields/invalid name/order. `orden` must be an integer and non-negative; `0` is accepted, negatives are rejected.

## 4.3 Subjects

### 24. `GET /api/subjects`

**Access:** Public / optional auth.

**Query parameters:**

| Parameter | Meaning |
| --- | --- |
| `q` | Search name, description or assigned professor names. |
| `careerId` | Filter by career. |
| `yearId` | Filter by career year. |
| `profesorId` | Filter by professor assignment in `subject_professors`. |

**Success:** `200 { "subjects": [ ... ] }`

Subject fields include:

`id`, `nombre`, `descripcion`, `profesorId`, `profesorNombre`, `profesores[]` (`id`, `nombre`, `apellido`, `nombreCompleto`), `careerYearId`, `estadoAsociacion`, `anioId`, `anioNombre`, `carreraId`, `carreraNombre`, `carreraColor`, `totalMateriales`, `createdAt`.

**Notes:** `profesorId` is the cached primary professor; `profesores[]` is authoritative. Ordering is career, year order, subject name.

### 25. `GET /api/subjects/:id`

**Access:** Public / optional auth.

**Success:** `200 { "subject": { ...with professors and counts... } }`

**Errors:** `404`.

### 26. `GET /api/subjects/:id/materials`

**Access:** Public / optional auth.

**Query parameters:**

| Parameter | Meaning |
| --- | --- |
| `q` | Search title/categories/description/author. |
| `tipo` | Material type. |
| `tags` | Repeat the parameter for multiple tags; all must match (`AND` on `categories LIKE`). A comma-separated string is treated as one literal tag here. |

**Success:** `200 { "subject": { ... }, "materials": [ ... ] }`

Only materials with `materials.estado = 'Aprobado'` **and** an `Aprobado` association with this subject are returned. `esFavorito` is added when authenticated.

**Errors:** `404` subject not found.

### 27. `POST /api/subjects`

**Access:** Superadmin.

**Input (JSON):**

| Field | Required | Rules |
| --- | --- | --- |
| `nombre` | yes | 2–100 characters. |
| `descripcion` | no | 0–500 characters. |
| `profesorId` | no | Existing user with role `Profesor`. |
| `profesorIds` | no | Array of existing professors; takes precedence over `profesorId`. |
| `careerYearId` | no | Existing `career_years.id`; must be a positive integer. |

**Action:** Creates subject with `estado_asociacion = 'Activa'`, optionally replaces professor assignments, records `Alta Materia` and one `Asociacion Materia` per assigned professor.

**Success:** `201 { "message": "...", "subject": { ... } }`

**Errors:** `404` professor/year not found; `422` invalid fields.

**Note:** A subject can be created without a professor; it is still `Activa`. Non-numeric or zero `profesorId` / `profesorIds` values are filtered out before validation, so they result in no assignment rather than a `422`; negative integer ids do trigger `422`.

## 28. `POST /api/subjects/:id/request-association`

**Access:** Profesor.

**Input:** none (the professor is `req.usuarioId`).

**Action:** Adds `subject_professors(subject_id, profesor_id)`, sets `subjects.profesor_id` to the caller and `estado_asociacion = 'Pendiente'`; records `Solicitud Materia`.

**Success:** `200 { "message": "Solicitud enviada..." }`

**Errors:** `404` subject; `403` from route if not a professor.

**Note:** The service does not verify that the professor exists/role; the route ensures the role. There is no exclusivity check: a subject can accumulate requests.

## 29. `POST /api/subjects/:id/confirm-association`

**Access:** Superadmin.

**Input:** optional `{ "profesorId": 2 }`.

**Action:** If `profesorId` is supplied, adds it to `subject_professors`; sets subject `Activa`; synchronizes `profesor_id` to the first assigned professor; records `Asociacion Materia`.

**Success:** `200 { "message": "Asociacion confirmada..." }`

**Errors:** `404` subject; DB errors if an invalid professor id violates the FK.

**Note:** This endpoint does not verify that `profesorId` has the `Profesor` role. A valid non-professor user id can be inserted into `subject_professors` through this path.

### 30. `POST /api/subjects/:id/reject-association`

**Access:** Superadmin.

**Input:** none.

**Action:** Deletes all `subject_professors` rows for the subject; sets `profesor_id = NULL` and sets `estado_asociacion = 'Pendiente'` only if it was already `Pendiente` (an active subject stays `Activa` but has no professors); records `Solicitud Materia`.

**Success:** `200 { "message": "Asociacion rechazada..." }`

**Errors:** `404` subject.

### 31. `PUT /api/subjects/:id`

**Access:** Superadmin.

**Input:** any of `nombre`, `descripcion`, `careerYearId`, `profesorId`, `profesorIds`.

**Action:** Updates provided fields; replaces professors when either professor field is present; records `Modificacion Materia`.

**Success:** `200 { "subject": { ... } }`

**Errors:** `404` subject/professor/year; `422` invalid name or year id.

**Notes:** The service accepts a no-op update (no fields) and still writes an audit row. `descripcion` is not length-validated on update. An empty `profesorIds` array removes all professors; non-numeric/zero professor ids are also filtered to an empty list, so they can remove assignments rather than producing a validation error. Negative integer ids trigger `422`.

## 4.4 Materials

### 32. `GET /api/materials/global`

**Access:** Public / optional auth.

**Query parameters:**

| Parameter | Meaning |
|---|---|
| `q` | Search title, categories, description, author first/last name. |
| `subjects` | Comma-separated or repeated subject ids; material must have an approved association with any of them. |
| `tags` | Comma-separated or repeated text; all must match `categories`. |
| `tipo` | Material type. |
| `careerId` | Filter by career of an approved association. |
| `yearId` | Filter by career year of an approved association. |

**Success:** `200 { "materials": [ ... ] }`

**Fields:** material fields plus `materias` (approved subject names), `materiasIds` (approved ids), `materiasTodosIds` (all association ids, including pending), `materiasDetalle`, `carreras`, and `esFavorito` (authenticated users).

**Important:** The global catalog includes **all approved materials**, even those with no subject associations. Subject pages require both `materials.estado = 'Aprobado'` and an approved association.

**Errors:** none for normal filters.

### 33. `GET /api/materials/author/:id`

**Access:** Public / optional auth.

**Query:** `q`, `tipo`, `tags`.

**Action:** Loads the author through `AuthService.getUserById` and returns only approved materials authored by that user.

**Success:** `200`

```
{
  "author": { "id": 2, "nombre": "...", "apellido": "...", "nombreCompleto": "...", "email":
"..." },
  "materials": [ ... ]
}
```

**Errors:** `404` author not found.

**Note:** The handler does not verify that the requested user is a material author or has a public role; any existing user id is returned with `nombre`, `apellido` and `email`. This exposes account email addresses through a public endpoint.

### 34. `GET /api/materials/mine`

**Access:** `Alumno`, `Profesor`, `Superadmin`.

**Query:** `q`, `tipo`, `tags`, `estado`.

**Success:** `200 { "materials": [ ... ] }` — all states for the current user, newest first; associations include pending/all ids.

**Errors:** `401`.

### 35. `GET /api/materials/pending`

**Access:** `Alumno`, `Profesor` (Superadmin is not allowed by the route).

**Success:** `200 { "materials": [ ... ] }` — current user's `Pendiente` materials only, newest first.

### 36. `GET /api/materials/pending-review`

**Access:** `Profesor`, `Superadmin`.

**Action:** Returns pending `material_subjects` rows in subjects where the current user is in `subject_professors`. Each row includes `subjectId` and `subjectNombre`.

**Success:** `200 { "materials": [ ... ] }`

**Note:** The query filters by `material_subjects.estado_aprobacion` only, not by `materials.estado`, so an already-approved material with a pending association (for example, a student duplicate) also appears. For a Superadmin who is not also assigned as a professor, this list is normally empty because the query filters by `sp.profesor_id = currentUserId`.

### 37. `GET /api/materials/favorites/mine`

**Access:** `Alumno`, `Profesor`, `Superadmin`.

**Query:** `q`, `tipo`, `tags`, optional `limit` / `offset` (passed through).

**Success:** `200 { "favorites": [ ...approved materials..., "esFavorito": 1 ] }`

**Errors:** `401`.

### 38. `POST /api/materials/upload`

**Access:** `Alumno`, `Profesor` (Superadmin cannot use this route; use batch upload).

**Content type:** `multipart/form-data`.

**Fields:**

| Field | Required | Rules |
|---|---|---|
| `titulo` | yes | 2–150 characters (frontend asks for 3+). |
| `descripcion` | no | Stored as trimmed text; empty becomes `NULL`. |
| `tipo` | conditionally | One of the 7 types; inferred from file/URL if omitted. |
| `archivo` | conditionally | Required when using a file type; max 100 MB. |
| `urlOrPath` | conditionally | Required when using a link type; validated by type. |
| `subjects` | yes | Comma-separated or repeated subject ids; at least one. |
| `categories` | no | Comma-separated or repeated tag text. |
| `portada` | no | Image; max 5 MB; valid image extension. |

**At least one** of `archivo` or `urlOrPath` must be effective after type inference.

**Action:**

- Professor: all selected subjects must be taught by the caller; material is saved as `Aprobado`; associations are `Aprobado`; subscribers are notified; audit `Subida Material`.
- Student: material is saved as `Pendiente`; associations are `Pendiente`; max 3 pending materials; no notification.
- Duplicate SHA-256:

    - Professor: existing material is associated to the subjects as `Aprobado`; subscribers notified; audit `Asociacion Material`; message says duplicate. The existing material's own `estado` is **not** changed by this path, so a pending duplicate does not become globally approved automatically.
    - Student: existing material is associated as `Pendiente`; pending limit applies; audit `Asociacion Material`. If the existing material is already `Aprobado`, it stays globally visible; only this subject association is pending.
- Categories are also inserted into `thematic_tags` with category `general`, ignoring failures for duplicates/banned tags.

**Success:** `201 { "message": "...", "material": { ... } }`

**Errors:**

| Status | Condition |
|---|---|
| `422` | Missing title/subject/file/link; invalid file/link/cover; title length. |
| `403` | Professor does not teach one of the selected subjects. |
| `409` | Student already has 3 pending materials. |
| `429` | Upload rate limit exceeded. |
| `400` | Multer file size/type errors. |

**Cleanup:** Newly written local files are deleted if the request fails after Multer. On a successful duplicate-hash upload, the newly uploaded file/cover is **not** removed; it remains as an orphan in storage (local or cloud) because the duplicate branch returns success without cleanup.

## 39. `POST /api/materials/batch`

**Access:** Superadmin.

**Content type:** `multipart/form-data`.

**Fields:**

| Field | Meaning |
|---|---|
| `archivo_0`, `archivo_1`, … or `archivos` | Files; indexed names preserve pairing. |
| `portada_0`, … or `portadas` | Optional covers. |
| `metadata` | JSON array of objects such as `[{ "titulo": "..." }]`. |
| `links` | Newline-separated `Titulo\|URL`, or an array/objects (`{titulo,url}`). |
| `categories` | Comma-separated tags applied to all items. |

**Limits:** 1–100 total items; max 220 uploaded files; 100 MB per file.

**Action:** Validates each file/link type, creates materials as `Aprobado` **without subject associations**, inserts submitted categories into `thematic_tags` (ignoring duplicate/banned failures), skips duplicate hashes (deleting the skipped file/cover through the storage helper), writes audit `Subida Material` per created

material, returns counts.

**Success:** `201 { "message": "...", "created": 2, "skipped": 1, "materials": [ ... ] }`

**Errors:** `422` no items, >100, invalid metadata/type/link/cover; `429`.

**Note:** Because materials have no subjects, they appear in the global catalog but not on subject pages until a professor associates them.

**Atomicity caveat:** `uploadBatch` processes items sequentially without a transaction. If a later item fails, earlier materials may already be committed, while the controller's local-mode error cleanup deletes every uploaded file/cover path collected for the request, potentially leaving those earlier DB rows pointing to removed files.

### 40. `POST /api/materials/:id/approve`

**Access:** `Profesor`, `Superadmin`.

**Input:** `{ "subjectId": 5 }` (required; converted with `Number`).

**Action:** Loads the subject; requires Superadmin or `subject.hasProfessor(caller)`; sets the `material_subjects` row to `Aprobado`; sets material `Aprobado`; notifies subscribers; records `Aprobacion Material`.

**Success:** `200 { "message": "Material aprobado..." }`

**Errors:** `404` subject; `403` not the professor; invalid `subjectId` becomes `NaN` and typically yields `404`.

**Note:** The service does not verify that the association row exists before updating; the normal UI only calls it for a pending row. It also does not verify that the material exists: with a valid subject and nonexistent material id, the UPDATEs affect zero rows, notification persistence fails silently, and the route still returns `200` and writes an approval audit row.

### 41. `POST /api/materials/:id/reject`

**Access:** `Profesor`, `Superadmin`.

**Input:** `{ "subjectId": 5 }`.

**Action:** Requires subject ownership (or Superadmin); deletes the association; counts remaining associations; if zero **and** the material is `Pendiente`, marks it `Rechazado`, clears URL/cover/hash, and deletes stored files; records `Cancelacion Material`.

**Success:** `200 { "message": "Asociacion rechazada...", "rejected": true|false }`

**Errors:** `404` subject/material; `403` not the professor.

**Note:** Rejecting one subject does not remove the material from other subjects. `rejected=true` only when the whole pending material is rejected. If the last association is removed from an already-`Aprobado` material, the material stays `Aprobado` with no subjects, its files are kept, and `rejected=false`.

### 42. `POST /api/materials/:id/associate`

**Access:** `Profesor` only.

**Input:** `{ "subjectIds": [5, 6] }` or `{ "subjectId": 5 }`.

**Action:** Requires role `Profesor`; loads material; normalizes ids; verifies every subject is taught by the caller; for each subject upserts an `Aprobado` association and notifies subscribers; records `Asociacion Material`.

**Success:** `200 { "message": "Material asociado...", "subjectIds": [5, 6] }`

**Errors:** `404` material; `403` not a professor/not teaching a subject; `422` missing/invalid ids.

**Note:** This endpoint can associate a pending material without changing its global `estado`; such a material still needs approval to be visible on subject pages.

## 43. `POST /api/materials/:id/cancel`

**Access:** `Alumno`, `Profesor` (Superadmin cannot use this route).

**Input:** none.

**Action:** Only the author (or Superadmin via service, but the route excludes Superadmin) can cancel, and only a `Pendiente` material. Sets `Cancelado`, clears URL/cover/hash, deletes associations, deletes stored files, records `Cancelacion Material`.

**Success:** `200 { "message": "Material cancelado..." }`

**Errors:** `404`; `403` not author; `409` material not cancellable (not pending).

## 44. `PUT /api/materials/:id`

**Access:** `Profesor`, `Superadmin`.

**Content type:** `multipart/form-data` (optional file/cover).

**Fields:**

| Field | Meaning |
|---|---|
| `titulo` | Optional; 2–150 if supplied. |
| `descripcion` | Optional; empty becomes `NULL`. |
| `tipo` | Optional; must be a valid type. |
| `categories` | Comma-separated or repeated tags. |
| `archivo` | New file; required for a file-type change if the current resource is a link/different file type. |
| `portada` | New cover image. |
| `removePortada` | `true` / `1` clears the cover when no new cover is sent. |
| `urlOrPath` | New link for link types. |

**Action:** Applies permission rules, validates transitions, checks duplicate hash, updates fields, creates new tags, deletes replaced files, records `Modificacion Material`.

**Success:** `200 { "message": "Material actualizado...", "material": { ... } }`

**Errors:** `404`; `403`; `409` duplicate hash; `422` no changes/invalid title/type/link/cover; `400` Multer errors.

**Permission summary:**

- Author can edit own material, but this route is limited to `Profesor` / `Superadmin`; a student author cannot call it.
- A professor can edit a student suggestion only when it belongs to a subject the caller teaches. Another professor's (or an administrator's) material is rejected before the subject-ownership check, even if the caller teaches that subject.
- Admin can edit student suggestions, but cannot edit another professor's or administrator's material unless the admin is its author (the non-student-author check rejects first).
- Student accounts are rejected by route middleware even when they authored the material.

### 45. `DELETE /api/materials/:id/subjects/:subjectId`

**Access:** `Profesor`, `Superadmin`.

**Action:** Removes only the association; requires the caller to teach the subject or be Superadmin; records `Desasociacion Material`. Does not delete the material, files or other associations.

**Success:** `200 { "message": "...", "deleted": false }`

**Errors:** `404` material/subject; `403`.

**Note:** A missing association still returns success and writes an audit row.

### 46. `DELETE /api/materials/:id`

**Access:** Superadmin.

**Input:** optional `?subjectId=5`.

**Action:**

- With `subjectId`: performs the same subject removal as #45 (admin passes the ownership check), audit `Desasociacion Material`.
- Without `subjectId`: deletes the material globally, cascades associations/favorites/notifications, deletes stored file/cover, audit `Borrado Material`.

**Success:** `200 { "message": "...", "deleted": true|false }`

**Errors:** `404` material/subject; `403` not applicable for route (admin only).

**Note:** Because this is a Superadmin route, a professor cannot use it for subject removal; use #45.

### 47. `POST /api/materials/:id/favorite`

**Access:** `Alumno`, `Profesor`, `Superadmin`.

**Input:** none.

**Action:** Toggles `material_favorites` for the current user and updates `materials.contador_favoritos`.

**Success:** `200 { "favorited": true|false, "contadorFavoritos": 4 }`

**Errors:** `404` material; the service's `400` invalid-id guard is normally unreachable through this route.

**Note:** Unknown materials are rejected, but any existing material state can be favorited; the favorites list later returns only approved materials. The favorite count is recomputed from `material_favorites`. The toggle is a read-then-write sequence with no explicit DB transaction, and the decrement is clamped at zero with `GREATEST(..., 0)`.

## 4.5 Subscriptions

### 48. `GET /api/subscriptions/mine`

**Access:** `Alumno`.

**Success:** `200`

```
{
  "subscriptions": [
    {
      "id": 1, "alumnoId": 3, "materiaId": 1, "estado": "Suscrito",
      "materiaNombre": "...", "materiaDescripcion": "...", "profesorId": 2,
      "created_at": "..."
    }
  ]
}
```

**Note:** Includes `Baneado` rows as well; the query does not filter by state. The repository returns `s.*` plus camel-case aliases, so the raw `alumno_id`, `materia_id` and `created_at` columns also appear; the timestamp is not renamed to `createdAt`.

### 49. `POST /api/subscriptions/subscribe/:subjectId`

**Access:** `Alumno`.

**Input:** none.

**Action:** Requires subject exists and `estado_asociacion = 'Activa'`; rejects if the existing subscription is `Baneado`; upserts `Suscrito`; records `Suscripcion Materia`.

**Success:** `201 { "message": "Suscripcion realizada..." }`

**Errors:** `404` subject missing/inactive; `403` banned; `401`.

### 50. `DELETE /api/subscriptions/subscribe/:subjectId`

**Access:** `Alumno`.

**Action:** Finds the subscription and deletes the row. No audit row is written.

**Success:** `200 { "message": "Suscripcion cancelada..." }`

**Errors:** `404` no subscription.

### 51. `GET /api/subscriptions/subject/:subjectId/observers`

**Access:** `Profesor`, `Superadmin`.

**Success:** `200`

```
{
  "observers": [
    { "id": 1, "alumnoId": 3, "materiaId": 1, "estado": "Suscrito", "email": "...", "nombre":
"...", "apellido": "..." }
  ]
}
```

Only `estado = 'Suscrito'` rows are returned.

**Notes:** The service does not verify that the caller teaches the subject. Any Professor/Admin can view any subject's observer list, and an unknown subject id returns an empty list rather than `404`. This is a privacy/validation gap to consider.

### 52. `POST /api/subscriptions/ban/:alumnoId/:subjectId`

**Access:** `Profesor`, `Superadmin`.

**Action:** Requires subject ownership (or Superadmin). If no subscription row exists, creates one with `Baneado`; otherwise updates it to `Baneado`. Records `Baneo Alumno` with the **student** as target.

**Success:** `200 { "message": "Alumno baneado..." }`

**Errors:** `404` subject; `403` not the professor.

**Note:** There is no API to unban a subject subscription; the student cannot re-subscribe while the banned row exists. The endpoint does not verify that `alumnoId` belongs to an `Alumno`; only the user FK is enforced. A nonexistent `alumnoId` can therefore surface as an untranslated foreign-key `500`.

## 4.6 Notifications

### 53. `GET /api/notifications`

**Access:** Any authenticated user.

**Success:** `200`

```
{
  "notifications": [
    {
      "id": 1, "subjectId": 5, "subjectNombre": "...", "materialId": 12,
      "materialTitulo": "...", "createdAt": "...", "readAt": null,
      "profesorNombre": "..."
    }
  ],
  "unreadCount": 3
}
```

**Notes:** Returns the latest 30 notifications, newest first. Notification rows are created only for `Suscrito` students when a professor publishes/associates material. The notification path does not check `materials.estado`, so it can point to a material that is still `Pendiente` and therefore not visible in the global

catalog or on subject pages.

### 54. `POST /api/notifications/read-all`

**Access:** Any authenticated user.

**Action:** Sets `read_at = NOW()` for all unread notifications of the caller.

**Success:** `200 { "read": true, "marked": 5 }`

### 55. `POST /api/notifications/:id/read`

**Access:** Any authenticated user.

**Action:** Marks one notification read if it belongs to the caller; idempotent (`COALESCE(read_at, NOW())`).

**Success:** `200 { "read": true }` even if no row matched.

**Errors:** `401`; no explicit `404`.

## 4.7 Audit

### 56. `GET /api/audit`

**Access:** Superadmin.

**Success:** `200 { "logs": [ ... ] }`

Each enriched log includes:

`id`, `tipoOperacion`, `usuarioId`, `registroId`, `timestamp`, `registroTipo`, `registroNombre`, `usuarioNombre`, `usuarioEmail`.

Ordering: newest first.

**Notes:** Target-name resolution uses the polymorphic `registro_id` and the operation category. Audit rows whose target was deleted show `null` for the label.

### 57. `GET /api/audit/user/:id`

**Access:** Superadmin.

**Success:** `200 { "logs": [ ... ] }` — raw rows where `usuario_id = :id`, newest first.

**Notes:** Does not check that the user exists; returns an empty list for unknown ids. This endpoint includes the raw rows plus the derived `registroTipo`, but it does not join/enrich actor or target names.

## 4.8 Thematic Tags

### 58. `GET /api/thematic-tags`

**Access:** Public.

**Query:** accepted but currently ignored by the repository.

**Success:** `200 { "tags": [ { "id": 1, "nombre": "...", "categoria": "general", "creadoPor": null, "baneado": 0, "createdAt": null } ] }`

Ordering: category, then name.

**Mapping caveat:** `findAll` / `findById` pass raw snake_case rows into `ThematicTag`, which reads only `creadoPor` and `createdAt`. List, ban and delete responses therefore report `creadoPor: null` and `createdAt: null` even when the database has values. `POST` returns the newly saved in-memory tag, which has `creadoPor` set but no `createdAt` property.

## 59. `POST /api/thematic-tags`

**Access:** `Alumno`, `Profesor`, `Superadmin`.

**Input:**

| Field | Required | Default |
|---|---|---|
| `nombre` | yes | — |
| `categoria` | no | `general` |

**Action:** Trims values. If `(nombre, categoria)` exists and is not banned, returns the existing tag; if it is banned, returns `400`; otherwise inserts the tag.

**Success:** `201 { "tag": { ... } }` (also for an existing non-banned tag).

**Errors:** `400` missing name or banned existing tag. The service does not enforce the 100-character column limits, so overlong `nombre` / `categoria` values can surface as an untranslated DB `500`.

**Audit:** None currently, despite the service constructor receiving `AuditLogService`.

## 60. `PUT /api/thematic-tags/:id/ban`

**Access:** Superadmin.

**Action:** Sets `baneado = 1`; returns the updated tag object (`{ ...tag, baneado: 1 }`). A banned tag cannot be created again but remains listed.

**Success:** `200 { "tag": { ... } }`

**Errors:** `404`.

**Audit:** None currently.

## 61. `DELETE /api/thematic-tags/:id`

**Access:** Superadmin.

**Action:** Deletes the tag row. Material `categories` text is untouched.

**Success:** `200 { "message": "ok" }`

**Errors:** `404`.

**Audit:** None currently.

# 5. Professor Action Playbook

| Action | Frontend | API call(s) | Service methods | DB effects | Audit |
|---|---|---|---|---|---|
| Log in | `/login` | `POST /api/auth/login` | `AuthService.login` | Read `users`, `roles` | — |
| Load dashboard | `/profesor` | `GET /api/subjects`, `GET /api/materials/pending-review`, `GET /api/materials/mine` | `SubjectService.getAllSubjects`, `MaterialService.getPendingForProfessor`, `getMyUploads` | Reads only | — |
| Publish material | Dashboard form | `POST /api/materials/upload` | `uploadOrSuggestMaterial` | `materials`, `material_subjects`, `thematic_tags`, `notifications`, `audit_logs` | `SubidaMaterial` |
| Publish duplicate file | Dashboard form | `POST /api/materials/upload` | `uploadOrSuggestMaterial` (duplicate branch) | `material_subjects`, `notifications`, `audit_logs` | `AsociacionMaterial` |
| Review suggestions | Pending panel | `GET /api/materials/pending-review`; `POST .../approve` or `.../reject` | `getPendingForProfessor`, `approveMaterial`, `rejectMaterial` | `material_subjects`, `materials`, `notifications`, `audit_logs` | `AprobacionMaterial` / `CancelacionMaterial` |
| Add existing material | Card modal | `POST /api/materials/:id/associate` | `associateMaterialToSubjects` | `material_subjects`, `notifications`, `audit_logs` | `AsociacionMaterial` |
| Edit material | Editor | `PUT /api/materials/:id` | `update` | `materials`, `thematic_tags`, `audit_logs`; old files deleted | `ModificacionMaterial` |
| View observers | Subject panel | `GET /api/subscriptions/subject/:id/observers` | `SubscriptionService.getObservers` | Reads `subscriptions`, `users` | — |
| Ban observer | Subject panel | `POST /api/subscriptions/ban/:alumnoId/:subjectId` | `banStudent` | `subscriptions`, `audit_logs` | `BaneoAlumno` |

| Action | Frontend | API call(s) | Service methods | DB effects | Audit |
|---|---|---|---|---|---|
| Remove from subject | Subject detail | `DELETE /api/materials/:id/subjects/:subjectId` | `delete(id, ctx, subjectId)` | `material_subjects`, `audit_logs` | `Desasociacion Material` |
| Request subject | API only | `POST /api/subjects/:id/request-association` | `requestAssociation` | `subject_professors`, `subjects`, `audit_logs` | `Solicitud Materia` |
| Confirm subject | Admin dashboard | `POST /api/subjects/:id/confirm-association` | `confirmAssociation` | `subject_professors`, `subjects`, `audit_logs` | `Asociacion Materia` |

### Professor upload quick rules

- Professor uploads are auto-approved.
- The professor must teach **every** selected subject.
- File hash duplicates do not create a second material.
- Subscribers receive notifications on publish/association.
- Student suggestions are pending and limited to three.
- A professor cannot publish through `/api/materials/batch`; that is admin-only.

## 6. Frontend Page Routes

| # | Method | Path | File/behavior | Access |
|---|---|---|---|---|
| 1 | `GET` | `/` | `frontend/html/index.html` (catalog) | Public |
| 2 | `GET` | `/catalogo` | `index.html` | Public |
| 3 | `GET` | `/materias` | `subjects.html` | Public |
| 4 | `GET` | `/login` | `login.html` | Public |
| 5 | `GET` | `/register` | `register.html` | Public |
| 6 | `GET` | `/forgot-password` | `forgot-password.html` | Public |
| 7 | `GET` | `/reset-password` | `302` redirect to `/forgot-password` | Public |
| 8 | `GET` | `/reset-password/:token` | `302` redirect to `/restablecer-password/:token` | Public |
| 9 | `GET` | `/recuperar` | `302` redirect to `/forgot-password` | Public |

| # | Method | Path | File/behavior | Access |
|---|---|---|---|---|
| 10 | `GET` | `/activar-cuenta/:token` | Calls `AuthService.activateAccount`, then `303` redirect to `/login?activacion=exitosa` or `=invalida` | Public |
| 11 | `GET` | `/restablecer-password/:token` | `reset-password.html`; no-store/no-referrer/noindex headers | Public |
| 12 | `GET` | `/confirmar-cuenta` | `302` redirect to `/login` | Public |
| 13 | `GET` | `/verify-account` | `302` redirect to `/login` | Public |
| 14 | `GET` | `/materia/:id` | `subject.html` | Public page; API enforces data access |
| 15 | `GET` | `/autor/:id` | `author.html` | Public page |
| 16 | `GET` | `/alumno` | `dashboard-alumno.html` | Page loads publicly; `authHelper` redirects unauthenticated users |
| 17 | `GET` | `/profesor` | `dashboard-profesor.html` | Same |
| 18 | `GET` | `/admin` | `dashboard-admin.html` | Same |
| 19 | `GET` | `/favoritos` | `favorites.html` | Same |
| 20 | `GET` | `/tests` | `tests.html` | Public test page |
| 21 | `GET` | unmatched non-file path | Sends `index.html` with status `404` | Public |
| — | `GET` | unmatched path containing `.` | `404` text `Recurso no encontrado` | Public |

**Important:** Page routes do not enforce authentication or roles. They serve HTML; the page controller performs client-side checks, and the API enforces authorization. A user can load `/admin` HTML, but admin API calls still return `403`.

## Test mode

When `NODE_ENV=testing`:

- `testsRoutes` is mounted at `/` and serves `tests.html` at `/`.
- `viewRoutes` (the friendly page routes above) is **not** mounted.
- API routes are still mounted normally.
- Upload rate limiting is disabled.
- `GET /` returns the manual test harness instead of the catalog.

### Static mounts

| Path | Serves |
|---|---|
| `/uploads/*` | Uploaded files/covers from `backend/uploads/`. |
| `/css/*`, `/js/*`, `/img/*`, `/html/*` | `frontend/` static assets via `express.static`. |
| `/html/index.html` | Static copy of the catalog page (the friendly `/` route serves the same file). |

`express.static` is mounted before the view routes, so existing files take precedence over page handlers.

# 7. Implementation Notes and Known Gaps

1. **JWT state staleness:** `verifyToken` trusts the token payload and does not re-query `estado`; account bans/deactivations take effect for new logins, not for already-issued tokens.
2. **Subject ownership:** `subject_professors` is authoritative; `subjects.profesor_id` is a cached primary professor.
3. **Global vs. subject catalogs:** `/api/materials/global` shows all approved materials, even unassigned ones. Subject pages require both material approval and an approved association.
4. **Duplicate professor upload:** associating an existing pending material as a professor does not change the material's global `estado`; only approval does.
5. **Approve safety:** `approveMaterial` updates association/material state without checking that the association or material exists. A valid subject with a nonexistent material id still returns `200` and writes an approval audit row. Normal UI flow uses valid pending rows.
6. **Observer privacy:** `GET /api/subscriptions/subject/:id/observers` does not check that the professor teaches the subject.
7. **No server pagination:** API list results are complete; the UI paginates.
8. **Audit coverage:** Tag operations are not audited. Unsubscribing is not audited. Login is not audited.
9. **Duplicate DB errors:** Some unique violations (career name, career-year name) are not translated and may return `500`.
10. **Empty updates:** `PUT /api/subjects/:id` accepts a no-op and still writes `Modificacion Materia`.
11. **Subject reject:** `reject-association` removes all professors but only changes `estado_asociacion` to `Pendiente` when it was already `Pendiente`.
12. **Material delete ambiguity:** The admin `DELETE /api/materials/:id` supports `?subjectId=` and then behaves like subject removal, not global deletion.
13. **Notification uniqueness:** `(usuario_id, subject_id, material_id)` is unique; re-publishing resets `read_at` and updates `created_at`.
14. **File cleanup:** Disk cleanup failures are logged; DB state may still change.
15. **Localized messages:** A service `message` key is translated by `ok()` only if it contains a dot. A literal message is returned as-is.
16. **Notifications can reference pending material:** professor association/duplicate paths notify subscribers without checking `materials.estado`.

17. **CORS and body parsing:** The server enables `cors()` with defaults and parses JSON/urlencoded bodies globally; multipart is handled per upload route.
18. **No refresh token:** Clients must log in again when the 2-hour JWT expires; `apiService` handles 401 by clearing the session.
19. **Default language:** `es`, unless `DEFAULT_LANGUAGE` is set.
20. **Test harness:** `NODE_ENV=testing` disables upload rate limiting and replaces page routes with the test harness at `/`.
21. **Role string exactness:** Authorization compares `Superadmin`, `Profesor`, `Alumno`; display translations do not change these values.
22. **Duplicate upload orphan:** a successful duplicate-hash upload leaves the newly stored file/cover in storage because cleanup runs only when the request fails in local mode.
23. **Public author email exposure:** `GET /api/materials/author/:id` returns name and email for any existing user id, without checking that the user authored materials or has a public role.
24. **No admin-account protection:** ban/unban/soft-delete accept any user id, including the caller or another Superadmin; there is no last-admin or self-protection rule.
25. **Batch partial failure:** `uploadBatch` is not transactional; a later item failure can leave earlier materials committed, while local-mode controller cleanup deletes every uploaded path collected for the request.